机器学习

传统学习和深度学习

计算机视觉领域：	自然语言处理领域：	其他领域：
1、图像识别	1、语言识别	1、智慧医疗
2、目标检测	2、机器翻译	2、推荐系统
3、自动驾驶	3、人机对话	3、智慧交通

环境





本课程使用的编程环境：

python==3.8
tensorflow==2.4.0
keras==2.4.3
numpy==1.19.5
pandas==1.3.5
matplotlib==3.4.2
sklearn==0.0

- 传统的是给出规则和数据，比如你要训练出一个模型，识别一个小猫什么什么的，你要给出有耳朵或者有其他特征的至少要求来训练模型
- 现在的深度学习是只需要给出数据和标签即可，训练集和验证集，模型就可以自己通过大量的数据训练后来判断一张从未见过的新图片

引入神经网络

- 神经网络不断分配权重和偏差w，b来达到一个满意的输出
- 神经网络过于复杂就会使模型变笨，死板，只会训练集里的东西，但是对新图片无好的判断能力，即为过拟合
- 欠拟合就是完全训练数据量都不够，根本没有训练出来

- 因此要找到一个合适而且能准确预测结果的模型成了一个重要的探索

计算机视觉领域：	自然语言处理领域：	其他领域：
1、图像识别	1、语言识别	1、智慧医疗
2、目标检测	2、机器翻译	2、推荐系统
3、自动驾驶	3、人机对话	3、智慧交通

线性回归问题

- 第一类研究的问题就是最简单的线性回归问题，特征量小容易用线性函数描述，f（x）=wx+b，要找到合适的w和b来使预测值fx和真实值y差距最小，这个差距MSE也就是损失，损失与x的函数关系就是损失函数Loss。MSE=abs【f（x）-y】
- 意思就是找到使损失函数接近0甚至完全为0的参数w和b就是我们的目标，为此我们开始探索方法，在算力逐渐强大的时代，穷举法变成了我们第一个idea，但是一旦当特征多了起来，穷举法需要的算力是非常庞大也是很浪费的。

最小二乘法

- 通过观察损失函数我们发现这是一个二次函数，使其最小就是求解这个数学表达式，对特征分别求偏导，也就延伸出第一个方法最小二乘法。这里有许多数学公式的推导涉及到矩阵和偏导等，但是我们发现最小二乘法还是不适于一些情况，有些特征矩阵是没有逆矩阵的，为此我们继续思考提出了一个十分重要的方法：梯度下降法

$$J(w)=\frac{1}{2}(w^{\top}X^{\top}Xw-Y^{\top}Xw-w^{\top}X^{\top}Y+Y^{\top}Y)$$

现在针对$J(w)$求导数，首先要知道如下的知识点：

$$\frac{\partial AB}{\partial B}=A^{T},\quad \frac{\partial A^{T}B}{\partial A}=B,\quad \frac{\partial X^{T}AX}{\partial X}=2AX$$

按这个规律对函数进行求导：

$$\begin{aligned}\frac{\partial J(w)}{\partial w}&=\frac{1}{2}\left(\frac{\partial w^{\top}X^{\top}Xw}{\partial w}-\frac{\partial Y^{\top}Xw}{\partial w}-\frac{\partial w^{\top}X^{\top}Y}{\partial w}\right)\\&=\frac{1}{2}[2X^{\top}Xw-(Y^{\top}X)^{\top}-(X^{\top}Y)]\\&=\frac{1}{2}[2X^{\top}Xw-2(X^{\top}Y)]\\&=X^{\top}Xw-X^{\top}Y\end{aligned}$$

令导数为$\frac{\partial J(w)}{\partial w}$=0，解得：

$$\begin{aligned}X^{\top}Xw-X^{\top}Y&=0\\X\top X&=X^{\top}Y\\W&=(X^{\top}X)^{-1}X^{\top}Y\end{aligned}$$

梯度下降法

现在假设有一个损失函数为下式：

$$J(w) = 4w^2$$

首先需要随机初始化$\boldsymbol{w}$,假设$\boldsymbol{w} = \mathbf{4}$，同时设定学习率为0.1。损失函数的导数如下式所示：

$$w_0 = 4, a = 0.1, \frac{\partial J(w)}{\partial w} = 8w$$

第一次$\boldsymbol{w}$更新的过程如下式计算：

$$\begin{aligned} w_1 &= w_0 - 0.1 \times \frac{\partial J(\omega)}{\partial w} \\ &= 4 - 0.1 \times 8 \times 4 \\ &= 0.8 \end{aligned}$$

后续$\boldsymbol{w}$更新的过程如下式：

$$\begin{aligned} W_2 &= 0.8 - 0.1 \times 8 \times 0.8 = 0.16 \\ W_2 &= 0.16 - 0.1 \times 8 \times 0.16 = 0.032 \\ W_4 &= 0.032 - 0.1 \times 8 \times 0.032 = 0.0064 \end{aligned}$$

假设此时设置的学习率为0.5：

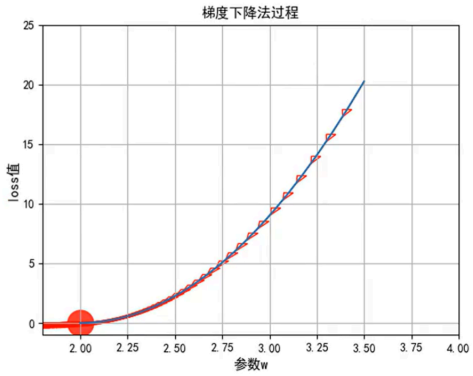
$$\begin{aligned} w_0 &= 4 \\ w_1 &= 4 - 05 \times 8 \times 4 = -12 \\ w_2 &= -12 - 05 \times 8 \times (-12) = 32 \\ w_3 &= 32 - 0.5 \times 8 \times 32 = 96 \end{aligned}$$

现在我们回到文章开头的小明学习成绩和学习时间关系的例子中来，看看怎么通过梯度下降法来进行最优$\boldsymbol{w}$的求解，这里还要注意的是，这里每次计算的梯度，是所有的数据的对应的梯度的平均值，计算公式如下所示：

$$\frac{J(w)}{\partial w} = \frac{1}{m} \frac{\partial \sum_{i=1}^{m} (f(x_i) - y_i)^2}{\partial w}$$

按照这个公式去计算的话，假设初始$\boldsymbol{w} = \mathbf{4}$，同时设定学习率为0.01

$$\begin{aligned} w_0 &= 4 \\ w_1 &= 4 - 0.01 \times \frac{1}{m} \frac{\partial J(w)}{\partial w} = 3.813 \\ w_2 &= 3.644 \\ w_3 &= 3.490 \\ &\vdots \\ w_{99} &= 2.000111 \end{aligned}$$



逻辑回归

- 逻辑回归其实跟二分类问题差不多，本质上并不是回归而是分类问题，只是利用回归的方法，利用sigmoid函数将函数映射到0-1的范围内然后做判断，设定阈值区分正例和反例，这时候也可以用概率来表示，非正即反。
- 实战思路也有变化，一般都是根据问题确定模型，找数据集（重要），梯度下降更新参数，定义损失函数，进行模型训练，最后训练完毕后进行模型评估优劣，一般来说要百分之九十以上才算比较良好的模型
- 在找数据集中，有非常重要的就是数据分析和数据预处理，并且根据情况来划分训练集和验证集，
- 模型评价指标也是依据问题而定。
- 学习好sklearn里面封装的函数是非常重要的。

在机器学习有监督学习中大致可以分为两大任务，一种是回归任务，一种是分类任务；那么这两种任务的区别是什么呢？按照较官方些的说法,输入变量与输出变量均为连续变量的预测问题是回归问题,输出变量为有限个离散变量的预测问题成为分类问题。

回归：

举个例子，输入一个人每日的运动时间、睡眠时间、工作时间、饮食等一些特征来预测一个人的体重，一个人的体重的值可以有无限个值。所以预测的结果是无限的、不确定的连续数值。这样的机器学习任务就是回归任务。

分类：

如果利用一个人每日的运动时间、睡眠时间、工作时间、饮食等一些特征来判断这个人的身体状况是否健康，那么最终的判断的结果就只有两种健康和不健康。这样的输出结果为离散值，预测的结果也是一个有限的数值来代表种类。

二分类的数学表达式

因此对于一个二分类的的问题，此时正例和反例的函数表达式就如下式所示：
预测结果为正例的表达式：

$$p(y=1|X)=\frac{1}{1+e^{-w^TX+b}}$$

预测结果为反例的表达式：

$$p(y=0|X)=\frac{e^{-w^TX+b}}{1+e^{-w^TX+b}}=1-p(y=1|X)$$

在函数的计算推导过程中，如果分别考虑正反两例情况，计算起来就特别麻烦，因此可以将上述两个例子合并起来得到如下公式：

$$p(y|X)=p(y|X)^y[1-p(y|X)]^{1-y}$$

逻辑回归的梯度下降

参数w参数更新：

参数w的偏导数的计算步骤如下所示：

$$\frac{\partial J_{(w,b)}}{\partial w} = -\frac{1}{m}\sum_{i=1}^{m}\frac{\partial\left(y_i \log\sigma(\omega^{\top}x_i+b)+(1-y_i)\log\left(1-\sigma(\omega^{\top}x_i+b)\right)\right)}{\partial w}$$

$$= -\frac{1}{m}\sum_{i=1}^{m} y_i \frac{\sigma(\omega^{\top}x_i+b)[1-\sigma(\omega^{\top}x_i+b)]}{\sigma(\omega^{\top}x_i+b)}x_i$$

$$+(1-y_i)\frac{-\sigma(\omega^{\top}x_i+b)[1-\sigma(\omega^{\top}x_i+b)]}{1-\sigma(\omega^{\top}x_i+b)}x_i$$

$$= -\frac{1}{m}\sum_{i=1}^{m} y_i x_i - \sigma(w^{\top}x_i+b)x_i$$

$$= \frac{1}{m}\sum_{i=1}^{m}(\sigma(w^{\top}x_i+b)-y_i)x_i$$

最终$\frac{\partial J_{(w,b)}}{\partial w}$的计算结果如下：

$$\frac{\partial J_{(w,b)}}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}(\sigma(w^{\top}x_i+b)-y_i)x_i$$

参数b参数更新：

参数b的偏导数的计算步骤如下所示：

$$\frac{\partial J_{(w,b)}}{\partial b} = -\frac{1}{m}\sum_{i=1}^{m}\frac{\partial\left(y_i \log\sigma(\omega^{\top}x_i+b)+(1-y_i)\log\left(1-\sigma(\omega^{\top}x_i+b)\right)\right)}{\partial b}$$

$$= -\frac{1}{m}\sum_{i=1}^{m} y_i \frac{\sigma(\omega^{\top}x_i+b)[1-\sigma(\omega^{\top}x_i+b)]}{\sigma(\omega^{\top}x_i+b)}$$

$$+(1-y_i)\frac{-\sigma(\omega^{\top}x_i+b)[1-\sigma(\omega^{\top}x_i+b)]}{1-\sigma(\omega^{\top}x_i+b)}$$

$$= -\frac{1}{m}\sum_{i=1}^{m} y_i\left[1-\sigma(\omega^{\top}x_i+b)\right]+(y_i-1)\sigma(\omega^{\top}x_i+b)$$

$$= \frac{1}{m}\sum_{i=1}^{m}\sigma(\omega^{\top}x_i+b)-y_i$$

最终$\frac{\partial J_{(w,b)}}{\partial b}$的计算结果如下：

$$\frac{\partial J_{(w,b)}}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\sigma(\omega^{\top}x_i+b)-y_i$$

实战常见步骤

1. 导入库：常见为numpy，pandas，matplotlib，进行数据预处理，数据分析清洗等

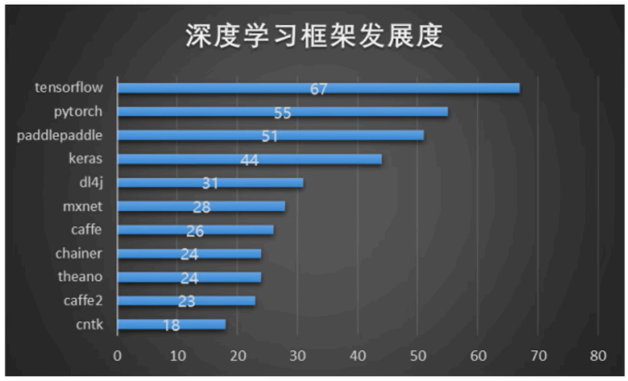
2. 导入框架，常见的就是sklearn
3. 导入数据dataset，用pandas去读取数据等
4. 因为特征和标签常常数值大小会影响后续模型训练，常常需要进行归一化去统一量纲，避免出现异常值影响训练
5. 数据处理之后进行模型搭建和正常梯度下降以及后续操作，最后进行模型评估并且将结果保存好。

深度学习DL

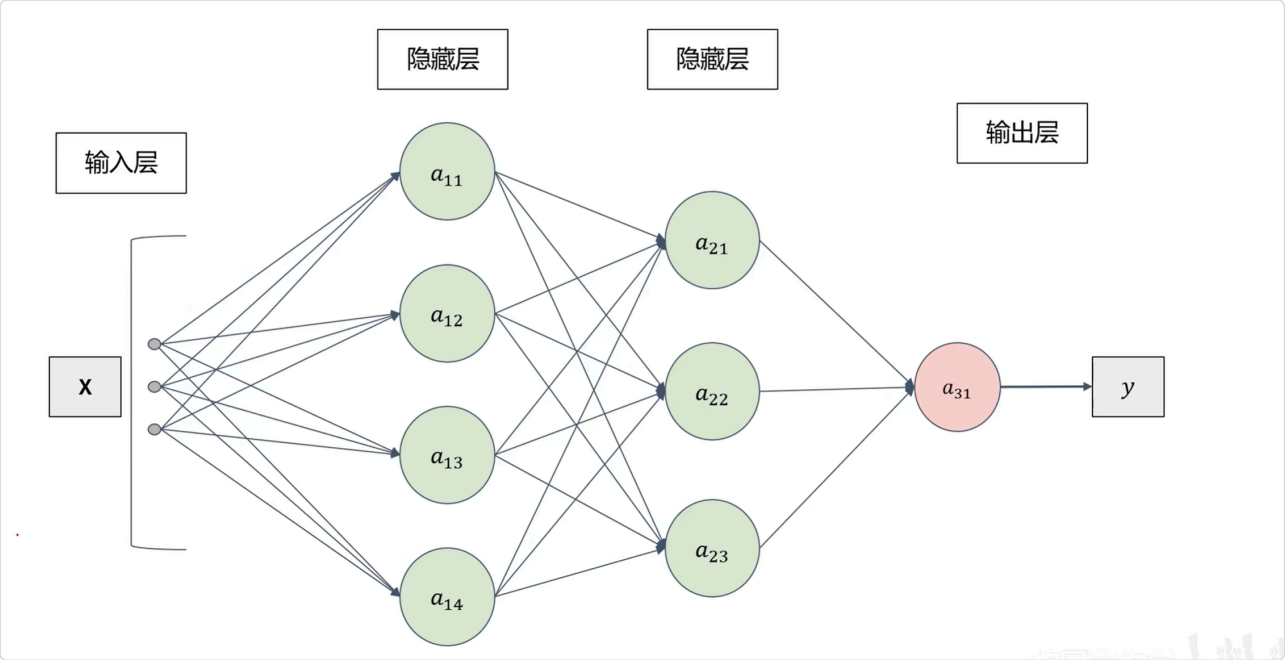
学习框架

深度学习框架





全连接神经网络

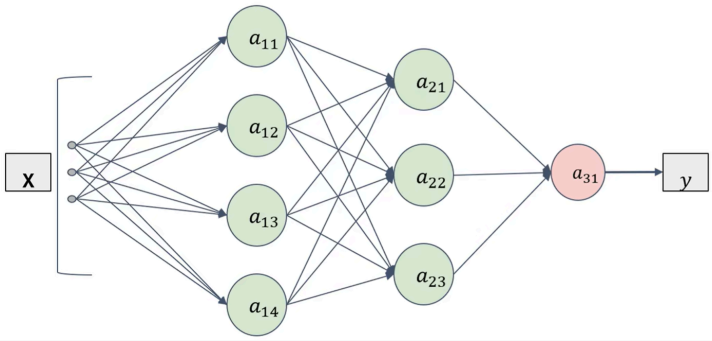


全连接神经网络：

不宜设置的太过于复杂，面对简单的问题容易过拟合，以结构单元为单位不断叠加。

为什么要加入激活函数：

类似于泰勒函数不断逼近，没有最好的激活函数，面对不同的问题有更合适的激活函数。



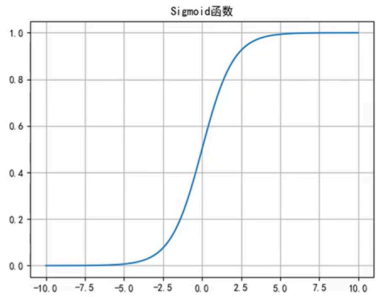
这里我们考虑把线性函数$h(x) = cx$作为激活函数，把$y(x) = h(h(h(x)))$的运算对应3层神经网络，这个运算会进行$y(x) = c * c * c * x$的乘法运算，但是同样的处理可以由$y(x) = ax$（注意，这里$a = c ** 3$）一次乘法运算（既没有隐藏层的神经网络）来表示。

常见的激活函数

Sigmoid

Sigmoid函数的公式和导数如式所示：

$$y = \frac{1}{1+e^{-z}} \Rightarrow y' = y(1-y)$$

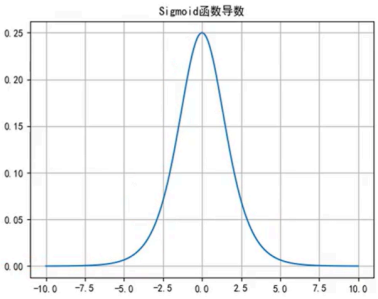


Sigmoid函数优点：

1、简单、非常适用分类任务；

Sigmoid函数缺点：

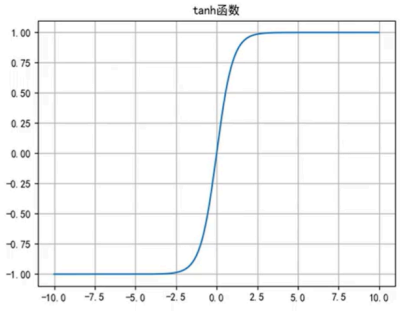
1、反向传播训练时有梯度消失的问题；
2、输输出值区间为(0,1)，关于0不对称；
3、梯度更新在不同方向走得太远，使得优化难度增大，训练耗时



Tanh

Tanh函数的公式和导数如式所示：

$$y = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \Rightarrow y' = 1-y^{2}$$

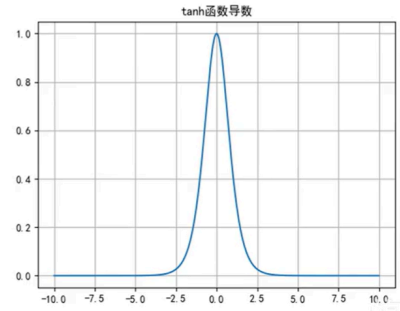


Tanh函数优点：

1、解决了Sigmoid函数输出值非0对称的问题
2、训练比Sigmoid函数快，更容易收敛；

Tanh函数缺点：

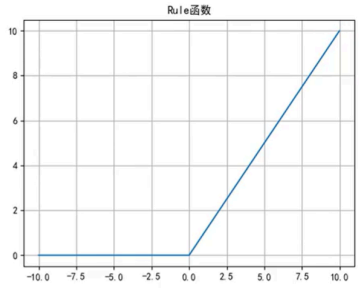
1、反向传播训练时有梯度消失的问题；
2、Tanh函数和Sigmoid函数非常相似。



Relu

ReLU函数的公式和导数如式所示：

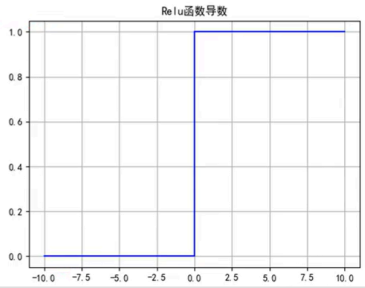
$$y=\begin{cases}z & \text{if } z>0\\ 0 & \text{if } z<=0\end{cases}\Longrightarrow y'=\begin{cases}1 & \text{if } z>0\\ 0 & \text{if } z<=0\end{cases}$$



ReLU函数优点：

1、解决了梯度消失的问题;

2、计算更为简单，没有Sigmoid函数和Tanh函数的指数运算;



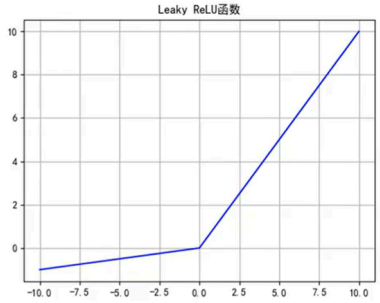
ReLU函数缺点：

1、训练时可能出现神经元死亡;

Leaky Relu

Leaky ReLU函数的公式和导数如式所示：

$$\mathrm{y}=\begin{cases}\mathrm{z} & \text{if } \mathrm{z}>0\\ \mathrm{az} & \text{if } \mathrm{z}<=0\end{cases}\Longrightarrow \mathrm{y}'=\begin{cases}1 & \text{if } \mathrm{z}>0\\ \mathrm{a} & \text{if } \mathrm{z}<=0\end{cases}$$

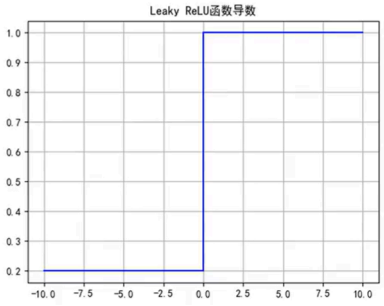


Leaky ReLU函数优点：

1、解决了ReLU的神经元死亡问题;

Leaky ReLU函数缺点：

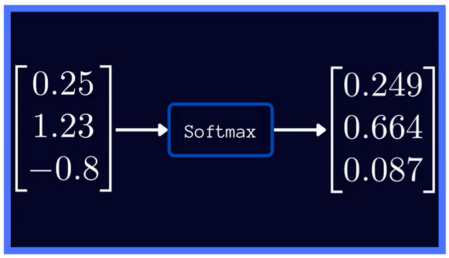
1、无法为正负输入值提供一致的关系预测（不同区间函数不同）



Softmax：用于多分类

Leaky ReLU函数的公式和导数如式所示：

$$y_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \Longrightarrow y_i' = y_i(1 - y_i)$$



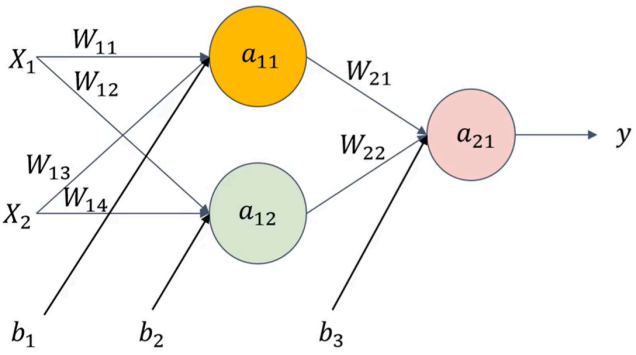
SoftMax函数的值域是在[0,1]之间的，并且存在多个输出，例如是一个3分类的任务，那么SoftMax函数最终的输出是对应每个类别的概率，同时这3个类别对应的概率相加最终的结果为1。因此在多分类任务的场景下，神经网络的最后一层一般都是使用SoftMax函数作为激活函数。

- 一般做输出层的激活函数

one hot编码

也叫热向量编码模式，应用于多分类场景，比如分成五类，若单纯表示为12345的，因为1+1=2，等出现相似可以相加的情况，会影响结果。 所以我们一般用向量的形式去表示分类，比如第一类[0,0,0,0,1]，第二类[0,0,0,1,0]以此类推，这样的话就不容易出现影响。并且也不会有第二类2大于第一类1的错误先验，虽然onehot编码必须要先确定数据分成几类。 后续可学习embedding嵌入编码更为主流：把类别映射到低维稠密向量。

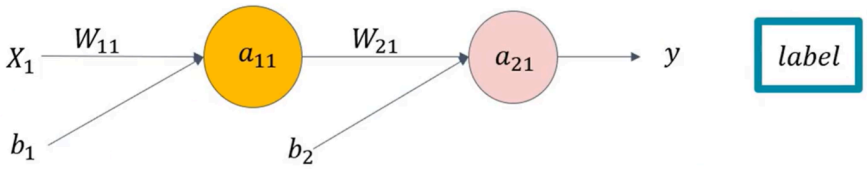
前向传播：即学习过程



前向传播计算过程：

$$a_{11} = \text{sigmoid}(x_1 w_{11} + x_2 w_{13} + b_1)$$
$$a_{12} = \text{sigmoid}(x_1 w_{12} + x_2 w_{14} + b_2)$$
$$a_{21} = \text{relu}(a_{11} w_{21} + a_{12} w_{22} + b_3)$$
$$y = a_{21}$$

每一个神经元都可能学到不同的特征。 前向传播具体计算过程：



前向传播计算公式：

$$a_{11} = \text{sigmoid}(x_1 w_{11} + b_1)$$
$$a_{21} = \text{relu}(a_{11} w_{21} + b_1)$$
$$y = a_{21}$$

具体计算过程：

初始化的值为$w_{11} = 0.5$，$w_{21} = 1$，$b_1 = 0.5$，$b_2 = 1$。初始化的值为$w_{11} = 0.5$，$w_{21} = 1$，$b_1 = 0.5$，$b_2 = 1$，$x_1 = 1$，label=2。

$$a_{11} = \text{sigmoid}(1 \times 0.5 + 0.5)$$
$$= 0.731$$
$$a_{21} = \text{relu}(0.731 \times 1 + 1)$$
$$= 1.731$$
$$y = 1.731$$

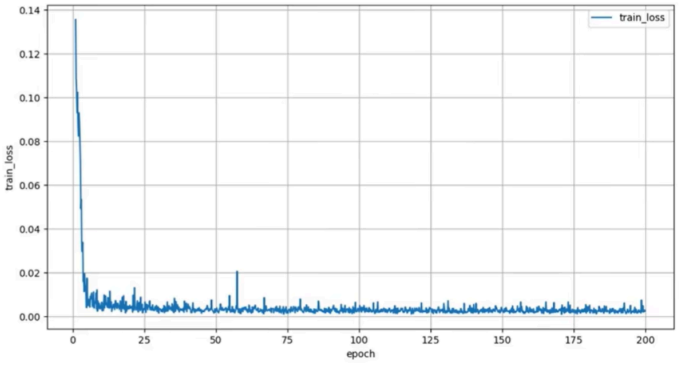
若已训练完成后，用训练好的模型去验证对象会得到新的预测值。

均方误差的损失函数：

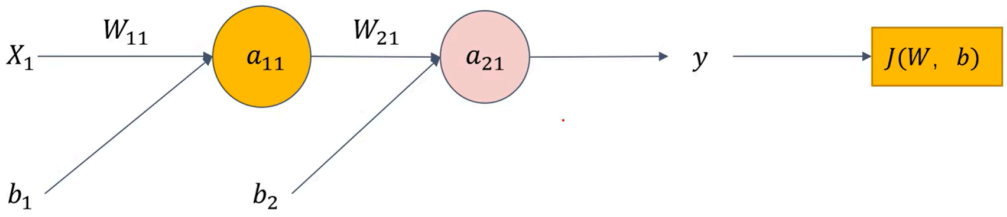
$$J(x) = \frac{1}{2m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

交叉熵损失函数：

$$J_{(x)} = -\frac{1}{m}\sum_{i=1}^{m}\left(y_i \log\sigma(w^{\top}x_i + b) + (1 - y_i)\log\left(1 - \sigma(w^{\top}x_i + b)\right)\right)$$



反向传播案例



案例

从图中可以看到该神经网络链一共有4个参数，分别为w_{11}，w_{21}，b_1，b_2，同时4个参数和输入值x_1分别有对应的初始值，分别为$w_{11}=0.5$，$w_{21}=1$，$b_1=0.5$，$b_2=1$，$x_1=1$。同时通过前向传播计算出来的$y=1.731$，label值为2。现在利用神经网络的前向传播计算出来的输出值和真实值label之间的误差进行反传播进行梯度更新。

对应参数的梯度：

$$\frac{\partial J(w_{21})}{\partial w_{21}}=-0.196639,\ \frac{\partial J(b_2)}{\partial b_2}=-0.269,$$
$$\frac{\partial J(w_{11})}{\partial w_{11}}=-0.052895,\ \frac{\partial J(b_1)}{\partial b_1}=-0.052895$$

保留3位小数方便计算：

$$\frac{\partial J(w_{21})}{\partial w_{21}}=-0.197,\ \frac{\partial J(b_2)}{\partial b_2}=-0.269,$$
$$\frac{\partial J(w_{11})}{\partial w_{11}}=-0.053,\ \frac{\partial J(b_1)}{\partial b_1}=-0.053$$

设置学习率大小为0.1：

$$w_{11}=0.5-0.1\times(-0.053)=0.505$$
$$w_{21}=1-0.1\times(-0.197)=1.020$$
$$b_1=0.5-0.1\times(-0.053)=0.505$$
$$b_2=1-0.1\times(-0.269)=1.027$$

整理一下：

$$w_{11}=0.505$$
$$w_{21}=1.020$$
$$b_1=0.505$$
$$b_2=1.027$$

新参数：

$$w_{11}=0.505$$
$$w_{21}=1.020$$
$$b_1=0.505$$
$$b_2=1.027$$

前向传播：

$$a_{11}=\text{sigmoid}(1\times0.505+0.505)=0.733$$
$$a_{21}=\text{relu}(0.733\times1.02+1.027)=1.775$$
$$y=1.775$$

结论：

此时最终的输出结果是$y=1.775$，可以看出这个数值比之前的前向传播输出值1.731要更加接近真实值2了

整体过程：

1. 数据预处理（数据清洗，数据集划分等操作）：这一部分其实一般占大头工作量

2. 确定模型
3. 训练
4. 计算误差
5. 计算梯度
6. 反向更新参数
7. 设置学习轮次
8. 导出模型
9. 验证模型

结论：

其实大体过程基本上是不会变化的，所谓创新点基本上不会更改流程，更可能去修改部分环节的创新，比如使用不同的训练模型，不同的数据处理方式，不同的计算方式等；

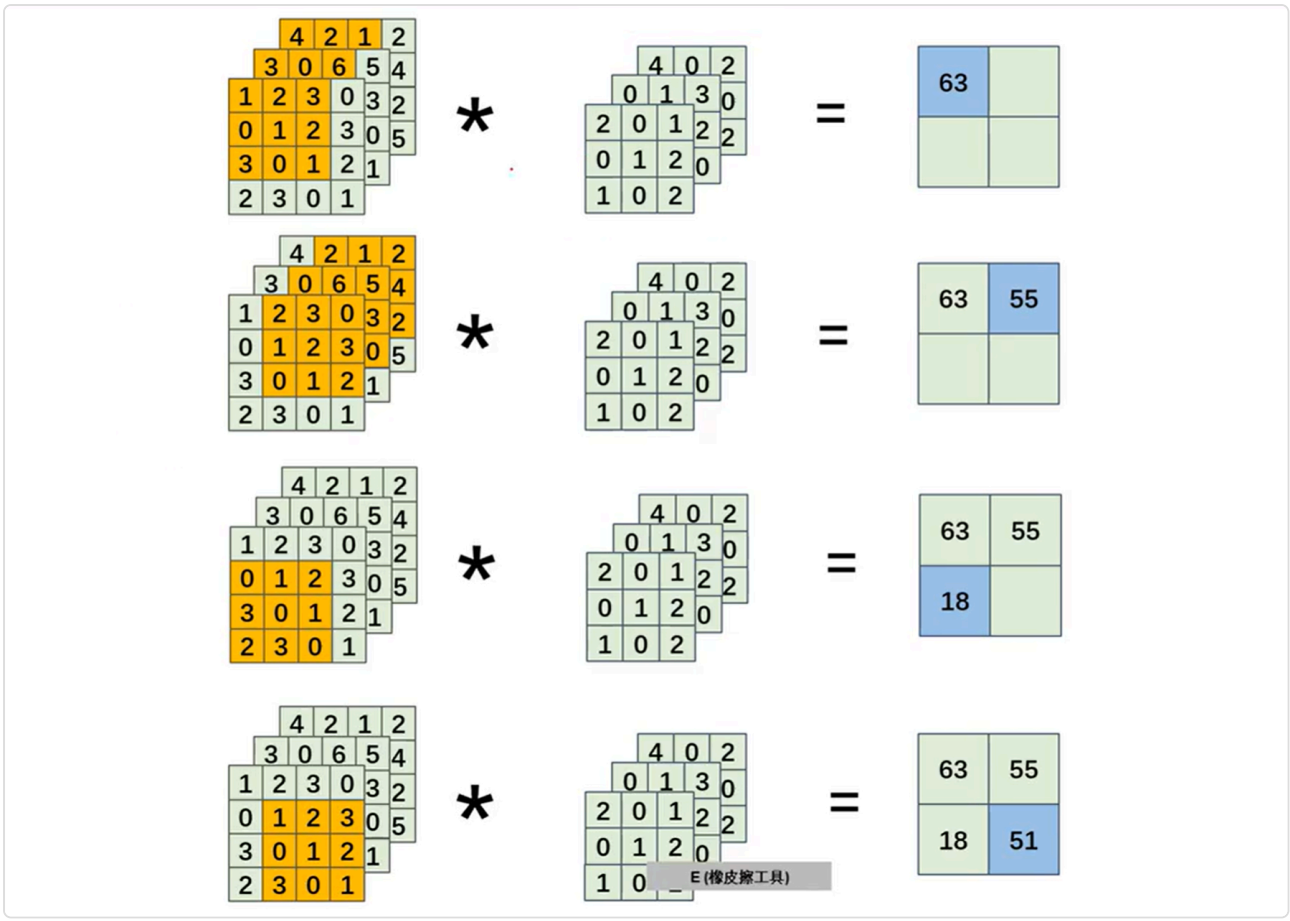
一般要先进行数据集的划分在进行类似于归一化和编码等操作，防止先操作后不好划分。

```
# 利用Keras框架帮助搭建深度学习网络模型
model = keras.Sequential()
model.add(Dense(10, activation='relu'))
model.add(Dense(10, activation='relu'))
model.add(Dense(2, activation='softmax'))
```

卷积神经网络（CNN）

1.图像的本质其实就是一个矩阵，矩阵的不同位置的数的大小就是亮度，一个图像可能有多个通道，经过相同位置的不同通道的数据叠加来展现出不同的像素观感。 视频的本身其实是图片的不断更换变化。

2.卷积神经网络一般包含：卷积，池化，全连接层三个环节： 卷积层：改变通道数，改变H，W 池化层：改变图像大小 全连接层：将处理好的进行特征输入，通过学习来进行输出 3.相较于全连接层：卷积神经网络有什么优势呢？ 全连接层如果输入图片必须要把图片展开成一个多行一列的矩阵，会丢失空间位置信息。并且参数量会过于庞大，容易造成过拟合现象。 4.比较出名的就是LENET，做缺陷检测

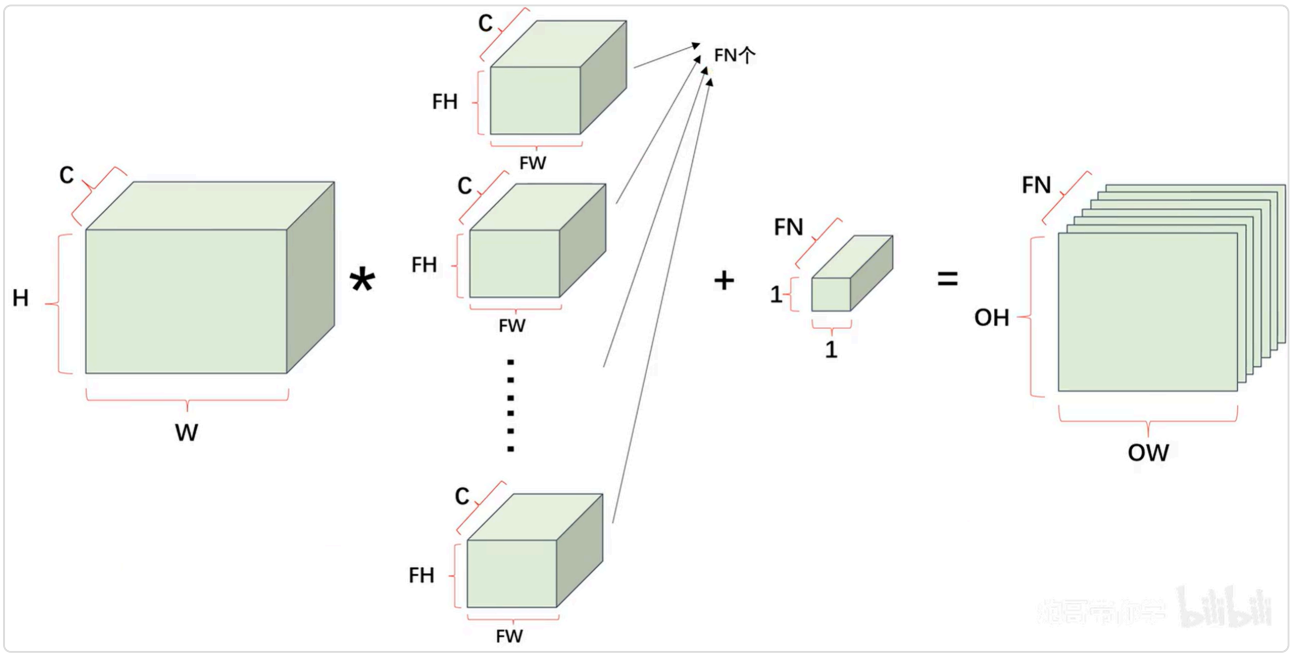


多通道运算

累加（各通道特征图*对应通道卷积核）=新特征图

权重共享：

滑块重复的块进行多次利用，不至于因为某个部分的亮度过高而有过大的权重去影响输出结果



池化层

一般有max池化和mean池化，其中max池化使用更多，会使模型更加健硕

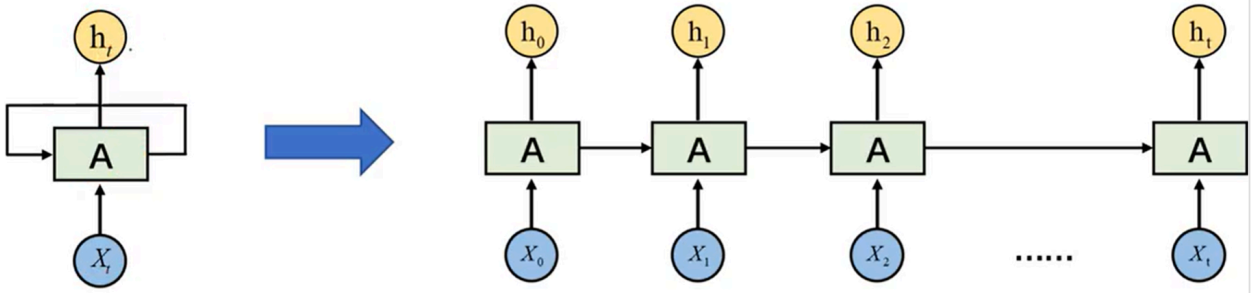
池化还有一些需要注意的地方：
输入的数据不仅有长和高，还有通道这一概念。但是池化层的运算不会对输入特征图的通道进行改变，池化操作是按通道独立进行计算的。
池化层对对微小的位置变化具有鲁棒性，使模型更加的健壮。当输入特征数据发现微小的变化的时候，输出的特征图的结果仍然是一样的，具体如下图所示：



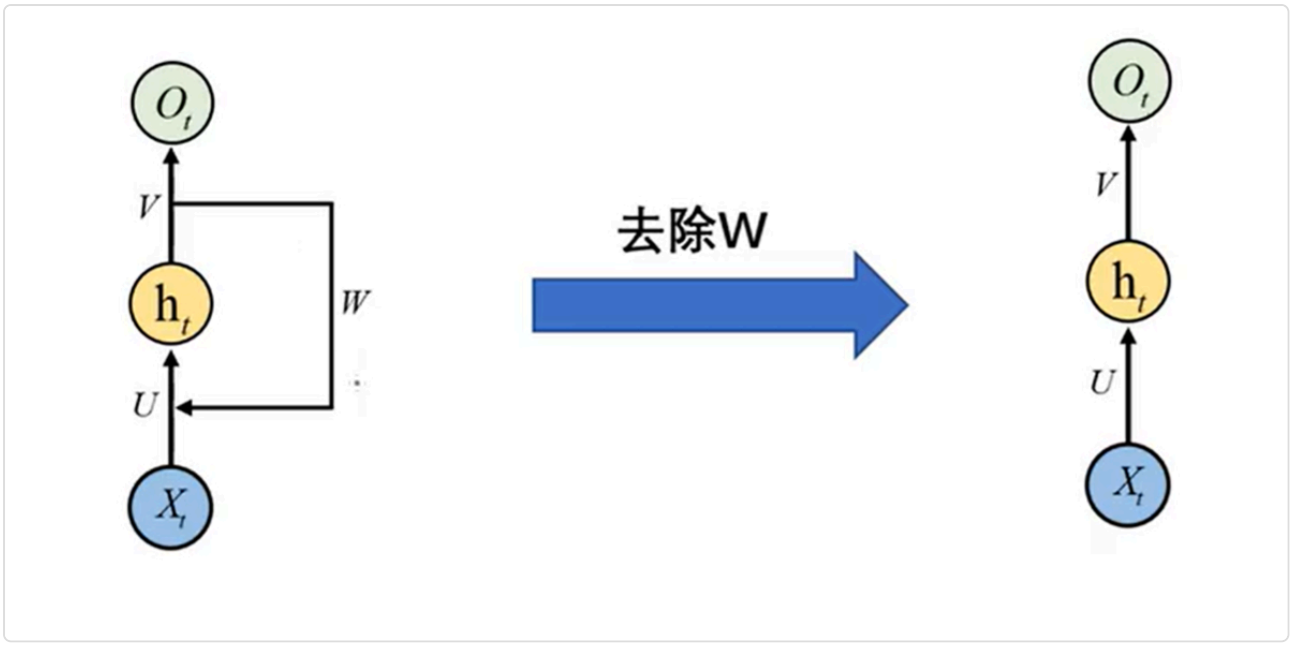
循环神经网络

1.分为RNN，LSTM，GRU等，最重要的思想就是前面的输入会对后面的输出有影响，可以记住顺序的一种神经网络，输入与时间序列有关。 2.前一层的隐藏层的输出做后一层的输入，下一时刻包含上一时刻的信息。

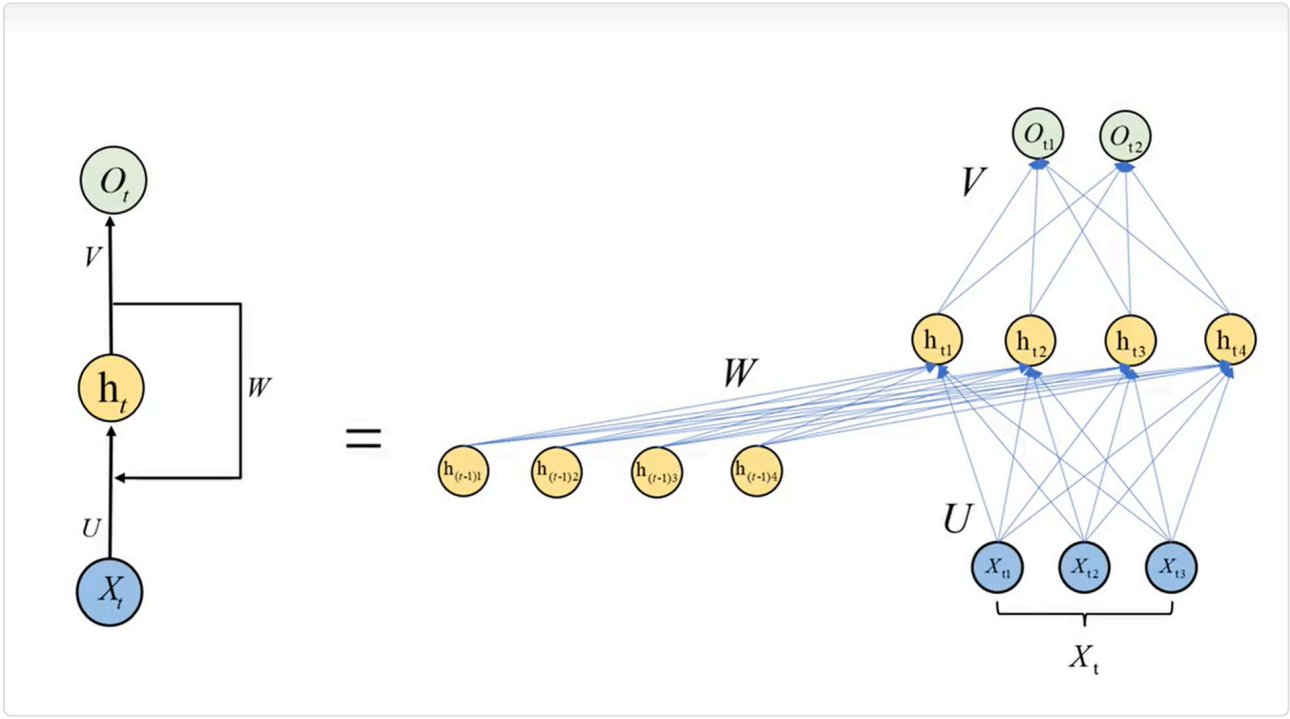
RNN结构：



RNN与全连接的区别：

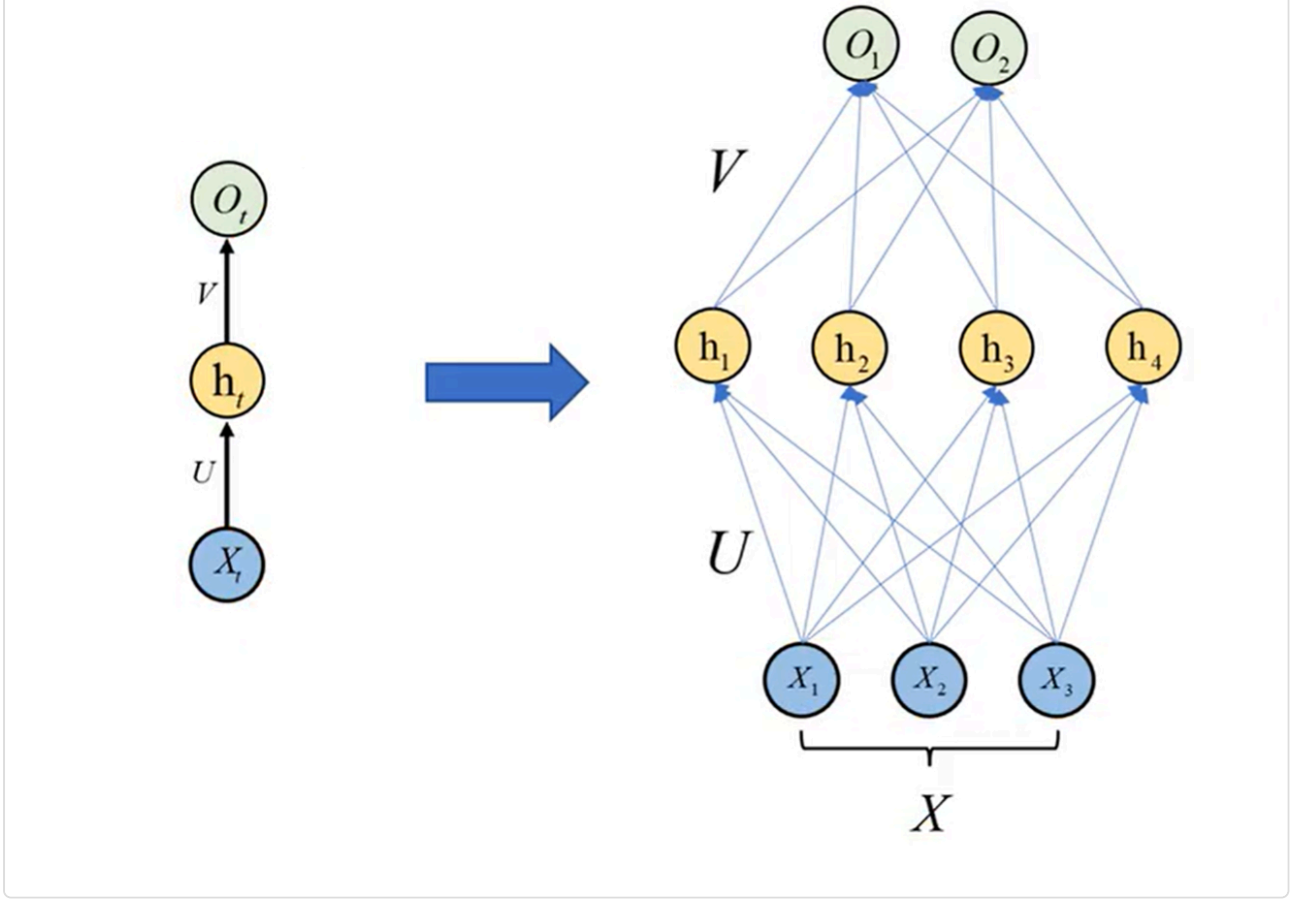


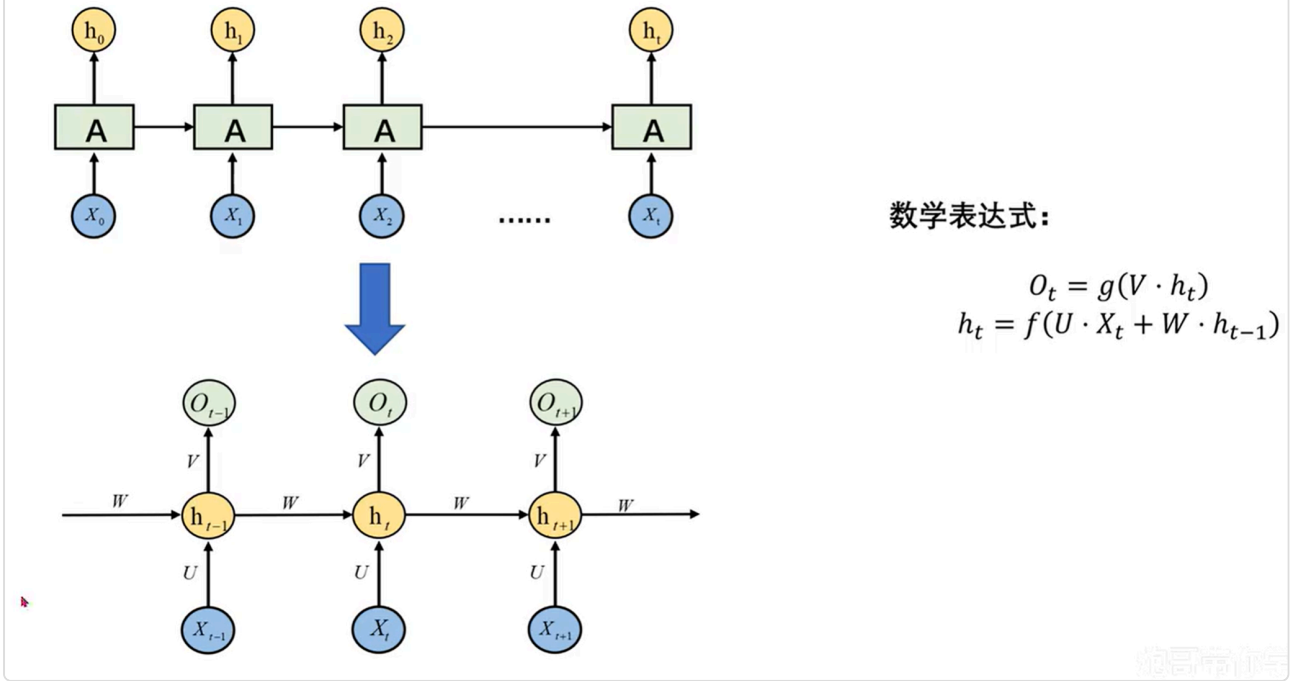
RNN运算过程：



权重共享：W，U，V在每一时刻权重都是一样的，有效的避免了过拟合的情况。 RNN存在梯度消失&梯度爆炸的问题。

全连接

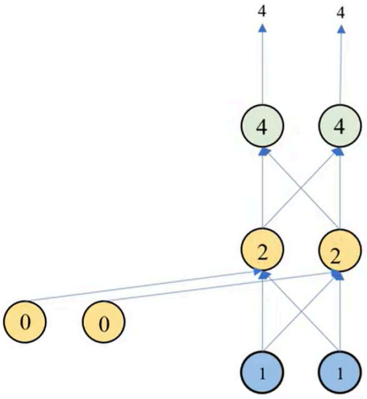




数学表达式：

$$O_t = g(V \cdot h_t)$$
$$h_t = f(U \cdot X_t + W \cdot h_{t-1})$$

前向传播案例：



设定神经网络的所有权重的大小都为1且没有偏置，激活函数也都是线性激活函数。同时在计算第一个时间步的输入的时候没有前一时刻的隐藏层输出值作为输入的，因此设定此时的隐藏层输出值为[0,0]

第1个神经元：

$$1\times1+1\times1+0\times1+0\times1=2$$

第2个神经元：

$$1\times1+1\times1+0\times1+0\times1=2$$

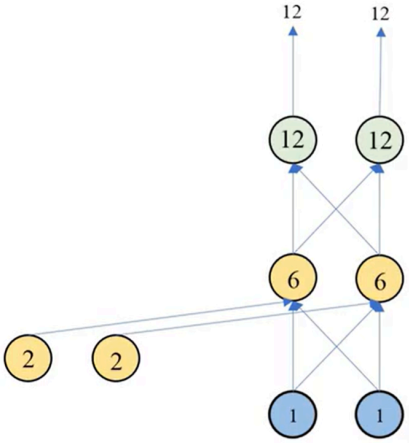
输出层第1个输出：

$$2\times1+2\times1=4$$

输出层第2个输出：

$$2\times1+2\times1=4$$

第二个时间部



第1个神经元：

$$1\times1+1\times1+2\times1+2\times1=6$$

第2个神经元：

$$1\times1+1\times1+2\times1+2\times1=6$$

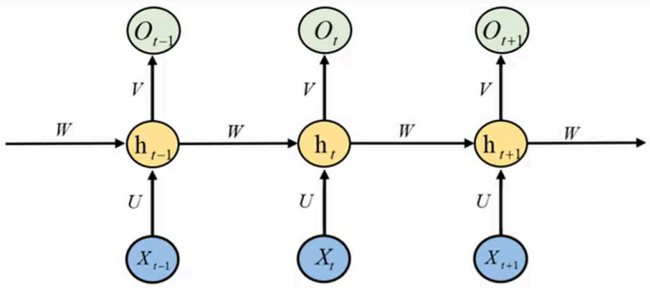
输出层第1个输出：

$$6\times1+6\times1=12$$

输出层第2个输出：

$$6\times1+6\times1=12$$

反向传播



RNN数学模型公式：

$$O_t = g(V \cdot h_t)$$
$$h_t = f(U \cdot X_t + W \cdot h_{t-1})$$

BPTT：

RNN神经网络中反向传播算法利用的是时间反向传播算法（BPTT），需要求解所有时间步的梯度之后，利用多变量链式求导法则求导求解梯度。

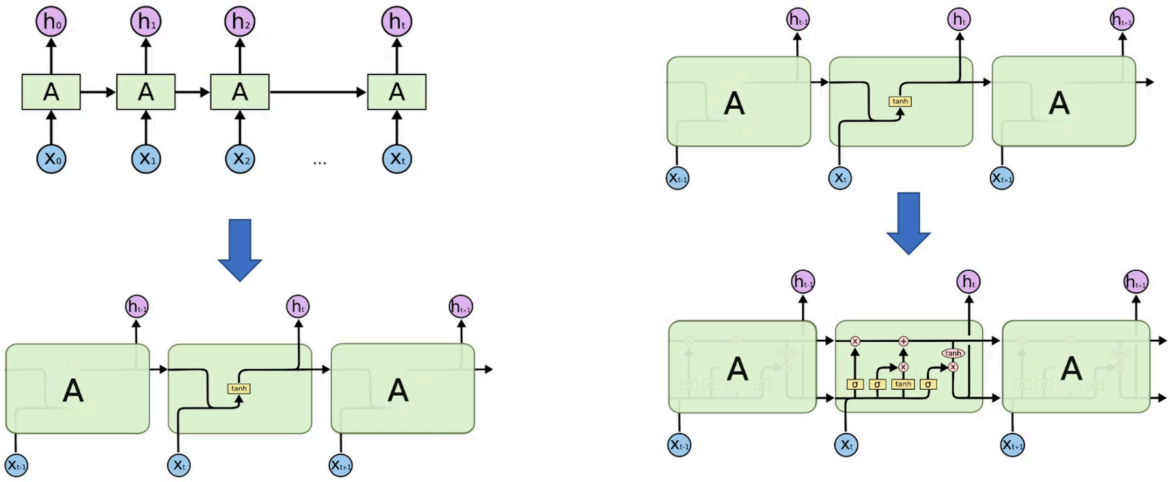
梯度计算公式：

$$\frac{\partial L_t}{\partial w} = \sum_{i=0}^{i=t} \frac{\partial L_t}{\partial 0_t} \frac{\partial 0_t}{\partial h_t} \frac{\partial h_t}{\partial h_i} \frac{\partial h_i}{\partial w}$$

LSTM

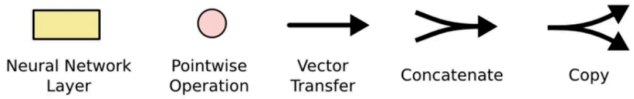
1.由三个门组成：输入门，遗忘门，输出门 输入门：输入保存当前有用信息 遗忘门：保留前一时刻的有用信息 输出门；输出当前门所获得的包含前面的所有有用信息 2.相较于RNN，LSTM能更好的去学习到有用的信息而不是把全部信息都保留了，不断更新细胞状态。

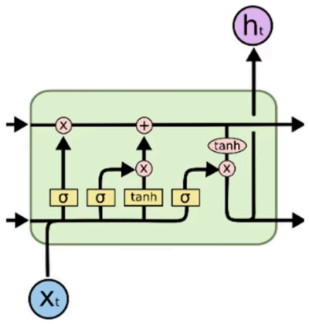
RNN和LSTM模型对比



三个门的运算过程都不同

LSTM模型基本运算





每一条黑线传输着一整个向量，从一个节点的输出到其他节点的输入。粉色的圈代表pointwise的操作，诸如向量的和，而黄色的矩阵就是学习到的神经网络层。合在一起的线表示向量的连接，分开的线表示内容被复制，然后分发到不同的位置。

